

LFCX:RAID-Systematik

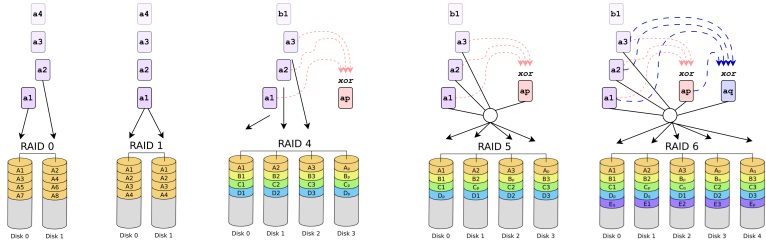
für die Ausbildung zur Fachinformatikerin

Karsten Reincke

GS-LDK

11. August 2026

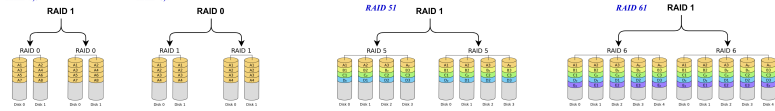
LFCX:RAID:Systematik



2 Platten der Größe Σ :	2 Platten der Größe Σ :	n Platten der Größe Σ :	n Platten der Größe Σ :	n Platten der Größe Σ :
Nutzdaten = 2Σ	Nutzdaten = Σ	Nutzdaten = $(n-1) \cdot \Sigma$	Nutzdaten = $(n-1) \cdot \Sigma$	Nutzdaten = $(n-2) \cdot \Sigma$
	Gesambedarf = 2Σ	Paritätsdaten = Σ	Paritätsdaten = Σ	Paritätsdaten = 2Σ
		Gesambedarf = $n\Sigma$	Gesambedarf = $n\Sigma$	Gesambedarf = $n\Sigma$

RAID 10
RAID 1+0
RAID 1,0

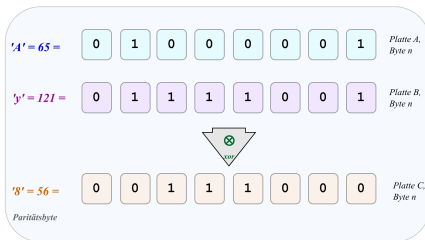
RAID 01
RAID 0+1
RAID 0,1



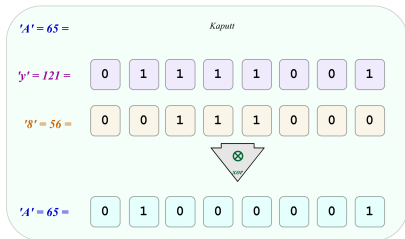
2n Platten der Größe Σ :	2n Platten der Größe Σ :
Nutzdaten = $(n-1) \cdot \Sigma$	Nutzdaten = $(n-2) \cdot \Sigma$
Paritätsdaten = 2Σ	Paritätsdaten = 4Σ
Gesambedarf = $2n\Sigma$	Gesambedarf = $2n\Sigma$

mit Bildern von <https://de.wikipedia.org/wiki/RAID>

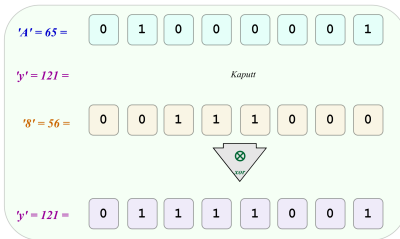
Sicherung



Restauration I



Restauration II



Woher wissen wir, dass

- für alle zwei beliebig gleich langen Bitzahlen
- mit XOR eine dritte Bitzahl erzeugt werden kann,
- aus der mit XOR und einer der beiden Zahlen die anderen rekonstruiert werden kann?

Woher wissen wir, dass

- für alle beliebig langen Ketten
- von gleich langen Bitzahlen
- mit XOR eine nächste Bitzahl erzeugt werden kann,
- aus der mit XOR und eine der Kettenzahlen rekonstruiert werden kann?

LFCX::Beweisverfahren

Deduktion(sbeweis)

Schlussregeln der
Deduktionsmaschine:

Modus Ponens

$$\begin{array}{l} S1: \forall x \ p(x) \rightarrow q(x) \\ S2: \underline{p(A)} \\ \hline \therefore \quad \quad \quad q(A) \end{array}$$

Modus Tollens

$$\begin{array}{l} S1: \forall x \ p(x) \rightarrow q(x) \\ S2: \underline{\neg q(A)} \\ \hline \therefore \quad \quad \neg p(A) \end{array}$$

Wissenbasis: $S1: \forall x \ hase(x) \rightarrow groß(ohren(x))$
 $S2: \underline{hase(ICH)}$
 Herleitung: $\therefore \quad \quad \quad groß(ohren(ICH))$

Wissenbasis: $S1: \forall x \ hase(x) \rightarrow groß(ohren(x))$
 $S2: \underline{klein(ohren(ICH))}$
 Herleitung: $\therefore \quad \quad \neg hase(ICH)$

$$\begin{array}{l} S1: \forall x \ hase(x) \rightarrow groß(ohren(x)) \\ S2: \underline{\neg hase(ICH)} \\ \hline \therefore \end{array} \quad /* \text{ keine konkrete Herleitung } */$$

$$\begin{array}{l} S1: \forall x \ hase(x) \leftarrow groß(ohren(x)) \\ /* \text{ Abduktion = keine Regel, keine Herleitung } */ \end{array}$$

Induktion(sbeweis)

- (1) Induktionsanfang: Beweise p für Fall 1
- (2) Induktionsvoraussetzung: Gesetzt, p gelte für Fall n
- (3) Induktionsschluss: Beweis p für Fall $n+1$ unter der Voraussetzung, dass p für Fall n gilt
- (4) Konklusio: p für alle (unendlich vielen Fälle)

Widerspruch(sbeweis)

- (1) Aufgabe: Beweise das p
- (2) Nimm an, dass $\neg p$
- (3) Beweis, dass unter der Bedingung p , Widerspruch!
- (4) Konklusio: p

unendlich viele Primzahlen



```
1  # (C) 2025 K.Reincke: proTirone snippet [CC-BY-4.0]
2
3
4
5  def diy(string,char):
6      return f"{char}{string}{char}"
7
8  print(diy(diy("", 'b'), 'a'))
9
```

Herzlichen Dank für Ihre Aufmerksamkeit!



Open Educational Resources

*Dieses Dokument stammt aus dem OER-Projekt <https://github.com/protirone/>, initiiert v. **Karsten Reincke, Hohenahr**. Es stellt **freie Unterrichtsmaterialien** samt **Quellcode** unter den Bedingungen der **CC-BY-4.0-Lizenz** zur Verfügung: Sie werden so wie sie sind angeboten, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).*

Bildnachweise

-  von Wikimedia Commons. **Lizenziert** unter **CC0**. Bereitgestellt auf **Wikimedia**.
-  von Karsten Reincke. **Lizenziert** unter **proTirone-Logo-Licence**. Bereitgestellt auf **github**. (may only be used as logo for proTirone)